```
# INPUT: only the octonion trilinear form T[i,j,k]=<e_i e_j, e_k> (Cayley-Dickson
doubling convention).
# QUESTION: what root/weight structure does the form ALONE produce? (no E8 assumed)
import numpy as np
np.random.seed(0); TOL=1e-9
def cd_mul(x,y):
    n=len(x)
    if n==1: return x*y
    h=n//2; a,b=x[:h],x[h:]; c,d=y[:h],y[h:]
    conj=lambda z: np.concatenate([z[:1],-z[1:]]) if len(z)>1 else z
    return np.concatenate([cd_mul(a,c)-cd_mul(conj(d),b),
cd_mul(d,a)+cd_mul(b,conj(c))])
E=np.eye(8); T=np.zeros((8,8,8))
for i in range(8):
    for j in range(8): T[i,j,:]=cd_mul(E[i],E[j])      # T[i,j,k]=<e_i e_j,e_k>
# basis of antisymmetric 8x8
Sb=[]
for p in range(8):
    for q in range(p+1,8):
        M=np.zeros((8,8)); M[p,q]=1; M[q,p]=-1; Sb.append(M)
# unknown (A,B,C) each antisymmetric: 84 params. invariance:
T(Ax,y,z)+T(x,By,z)+T(x,y,Cz)=0
cols=[]
for s in range(3):
    for M in Sb:
        if s==0: r=np.einsum('ai,ajk->ijk',M,T)
        if s==1: r=np.einsum('bj,ibk->ijk',M,T)
        if s==2: r=np.einsum('ck,ijc->ijk',M,T)
        cols.append(r.ravel())
Msys=np.array(cols).T
u,sv,vt=np.linalg.svd(Msys); null=vt[(sv<TOL).sum() if False else np.sum(sv>TOL):]
d=null.shape[0]
print("Lie algebra preserving T: dimension",d,"(smallest kept sv %.2e, largest
dropped %.2e)"%(sv[sv>TOL].min(), sv[sv<=TOL].max() if (sv<=TOL).any() else 0))
def trip(v):
    out=[]
    for s in range(3):
        M=np.zeros((8,8))
        for k,B in enumerate(Sb): M+=v[s*28+k]*B
        out.append(M)
    return out
L=[trip(v) for v in null]; flat=lambda t:np.concatenate([m.ravel() for m in t])
# coordinates in algebra basis (null rows orthonormal in the 84-param space)
def coords(t):
    v=np.concatenate([np.array([M[p,q] for p in range(8) for q in range(p+1,8)]) for
M in t]); return null@v
```

```
def comm(X,Y): return [X[s]@Y[s]-Y[s]@X[s] for s in range(3)]
closure=max(np.linalg.norm(flat(comm(L[a],L[b]))-
flat(trip(null.T@coords(comm(L[a],L[b])))) ) for a in range(d) for b in range(d))
print("closure under commutator, max residual: %.2e"%closure)
ad=lambda X: np.array([coords(comm(X,L[b])) for b in range(d)]).T
X=trip(np.random.randn(d)@null)
adX=ad(X); u2,s2,v2=np.linalg.svd(adX); cent=v2[np.sum(s2>1e-9):]
print("centralizer of generic element: dim",cent.shape[0],"(= rank)")
Hs=[trip(c@null) for c in cent]
adH=[ad(H) for H in Hs]
K=np.array([[np.trace(a@b) for b in adH] for a in adH]); print("Killing form on
Cartan eigenvalues (all negative => compact):",np.round(np.linalg.eigvalsh(K),4))
Hg=sum(np.random.randn()*a for a in adH)
w,V=np.linalg.eig(Hg); nz=np.abs(w)>1e-9
print("ad-eigenvalues: zero:",(~nz).sum()," nonzero (roots):",nz.sum())
srt=np.sort(np.abs(w[nz].imag)); print("min gap between distinct root eigenvalues
%.3e"%np.min(np.diff(np.sort(w[nz].imag))))
def functionals(mats,vecs):   # joint eigenvalue vectors a_i=Im(v^H H_i v/v^H v)
    return np.array([[ (v.conj()@m@v/(v.conj()@v)).imag for m in mats] for v in
vecs.T])
Kinv=np.linalg.inv(-K)
R=functionals(adH,V[:,nz]); nrm=np.einsum('ni,ij,nj->n',R,Kinv,R)
print("root norms (Killing metric): min %.6f max %.6f"%(nrm.min(),nrm.max()))
sc=2/nrm.mean(); gram=sc*np.einsum('ni,ij,mj->nm',R,Kinv,R)
print("root-root products (roots scaled to
length^2=2):",sorted(set(np.round(gram.ravel(),6))))
# weights of the three 8-dim reps
Wt={}
for s,name in enumerate(["8_v (A)","8_s (B)","8_c (C)"]):
    mats=[H[s] for H in Hs]; Hs_g=sum(np.random.randn()*m for m in mats)
    ww,VV=np.linalg.eig(Hs_g); F=functionals(mats,VV); Wt[name]=F
    n=sc*np.einsum('ni,ij,nj->n',F,Kinv,F)
    print(name,": 8 weights, norm^2 values",sorted(set(np.round(n,6))),", distinct
weights:",len(set(map(tuple,np.round(F,6)))))
```